

Signal Processing, Machine Learning and Control HW1

Sravanth Chowdary Potluri
und5uv
CS6762

September 9, 2025

1. Enabling HW/SW Technologies for CPS/IoT

The Cyber-Physical Systems (CPS) and Internet of Things (IoT) revolution is supported by a combination of advancing hardware and software technologies.

Hardware (HW) technologies include:

- **Microcontrollers (MCUs):** Low-cost, low-power processors that act as the "brain" of a smart device.
- **Sensors:** Devices that measure physical phenomena and convert them into electrical signals. There are hundreds of types, such as those for temperature, light, and motion.
- **Actuators:** Components that perform a physical action based on a signal, such as turning on a light or moving a valve.
- **Memory:** Various types of memory are used, including RAM for active processing, Flash for storing program code, and EEPROM for non-volatile data storage.
- **Wireless Transceivers:** Radios and antennas that enable communication with other devices and the internet.
- **Energy Sources:** Since devices are often untethered, they rely on power sources like batteries, supercapacitors, or energy scavenging technologies like solar cells.

Software (SW) technologies include:

- **Programming Languages and APIs:** High-level languages like Python and C++ are used on platforms like Raspberry Pi and Arduino, with specific APIs to interact with sensor hardware.
- **Signal Processing (SP):** A set of algorithms, such as Convolution and the Discrete Fourier Transform (DFT), used to process raw data from sensors to extract useful information.
- **Machine Learning (ML):** Algorithms like decision trees and deep neural networks (DNNs) that allow systems to learn from sensor data and make intelligent decisions or classifications.

2. Hardware Components of a Smart Device

A typical smart device, also known as a sensor node, integrates several hardware components to enable it to sense, process, and communicate. A complete list of these components includes:

- **Microcontroller (MCU):** The central processing unit that executes programs.
- **Memory:** Includes RAM, Flash, and EEPROM for data and program storage.
- **Sensors:** One or more sensors to gather data from the physical world (e.g., light, temperature, acceleration).
- **Actuators:** Components to affect the physical world (e.g., a buzzer or an LED).
- **Transceivers and Antennas:** To send and receive data wirelessly.
- **Power Source:** Typically a battery, but can also include supercapacitors or solar cells for energy harvesting.
- **Busses:** Standardized communication interfaces like SPI and I2C used to connect components like sensors to the microcontroller.
- **Displays:** Some devices may include screens for outputting information to a user.

3. Energy Scavenging

Energy scavenging, also known as energy harvesting, is the process by which a device captures small amounts of energy from its surrounding environment and converts it into usable electrical power. This allows devices to operate for extended periods without needing their batteries replaced.

A common example shown in the slides is the use of **solar cells** to capture energy from light. This scavenged energy can be used to power the device directly or to charge a battery or supercapacitor. This is a critical technology for deploying long-lasting, low-maintenance IoT sensor networks.

4. Properties of Sensors

When choosing a sensor for an application, there are **14 main properties** that must be considered to ensure it meets the requirements. These properties are:

1. **Range:** The minimum and maximum values the sensor can measure.
2. **Accuracy:** The measure of error and uncertainty in the sensor's readings.
3. **Repeatability:** The ability of the sensor to produce the same output for the same input over multiple trials.
4. **Linearity:** How closely the sensor's output follows a linear relationship to the input.

5. **Sensitivity:** The ratio of change in the output signal to a change in the input physical property.
6. **Efficiency:** The ratio of the output power to the input power.
7. **Resolution:** The smallest change in the input that the sensor can detect and report.
8. **Response Time:** How quickly the sensor's output reflects a change in the input.
9. **Overshoot:** How much the output signal exceeds the expected value when the input changes rapidly.
10. **Drift and Stability:** How the sensor's output signal varies slowly and independently of the input over time.
11. **Offset:** The output value of the sensor when there is no input.
12. **Packaging:** The physical housing and form factor of the sensor.
13. **Power Drain:** The amount of power the sensor consumes during operation.
14. **Initialization Time:** The time it takes for a sensor to be ready to take accurate readings after being powered on, which is important for power-saving sleep cycles.

5. Nyquist Sampling Theorem

The **Nyquist sampling theorem** (also known as the Nyquist-Shannon sampling theorem) is a fundamental principle in signal processing. It states that in order to accurately reconstruct a continuous analog signal from a series of discrete samples, the **sampling frequency (f_s) must be strictly greater than twice the maximum frequency (f_{max}) present in the signal.**

Mathematically, this is expressed as: $f_s > 2f_{max}$

If this condition is met, the original signal can be perfectly captured without loss of information. If it is not met, an error known as aliasing will occur. For example, to properly sample a signal with frequencies up to 500 Hz, the sampling rate must be above 1000 Hz.

6. Aliasing and How to Avoid It

Aliasing is an effect that causes high-frequency signals to be misrepresented as lower-frequency signals when sampled at a rate that is too low. When the sampling frequency is less than twice the signal's maximum frequency, the samples collected are insufficient to capture the true shape of the wave, resulting in a distorted, "aliased" signal. The lecture slides illustrate this by showing a 900 Hz signal incorrectly appearing as a 100 Hz signal when sampled at 1000 Hz.

To **avoid aliasing**, you must adhere to the **Nyquist sampling theorem**. This means you must ensure that the sampling frequency is at least twice the highest frequency component of the signal you intend to measure. Another common practice is to use an analog low-pass filter (an anti-aliasing filter) before the ADC to remove any frequencies above half the sampling rate.

7. Virtual Sensors

A **virtual sensor** is a software-based abstraction that produces data by combining and processing information from one or more physical hardware sensors. It doesn't measure a physical property directly; instead, it infers a new, often more complex or contextual piece of information.

The concept can be summarized as *Hardware + Software = Sensor*. For example, a physical **magnetometer** measures magnetic field strength, but software can process this data to create a virtual **compass** sensor that provides a heading in degrees. Other examples include a "fall detection sensor" that fuses data from an accelerometer and a gyroscope, or a "gesture detection sensor" that interprets data from muscle activity sensors.

8. Typical ADC Parameters

An Analog-to-Digital Converter (ADC) is a critical component that converts continuous analog signals from sensors into discrete digital values for a microcontroller. The two main parameters that define an ADC's performance are:

1. **Resolution:** This determines the precision of the conversion and is typically measured in bits. A higher bit count means the ADC can represent the analog signal with more discrete steps. For example, a 10-bit ADC can represent a signal with 2^{10} or 1024 distinct values, while a 12-bit ADC can represent it with 2^{12} or 4096 values.
2. **Sample Rate:** This is the speed at which the ADC can perform conversions, measured in samples per second (SPS) or Hertz (Hz). The required sample rate depends on the maximum frequency of the signal being measured, according to the Nyquist theorem.

Other parameters include the number of input channels the ADC can handle and its power consumption.

9. Connecting Peripherals to Microcontrollers

Peripherals and sensors are connected to microcontrollers primarily through the microcontroller's **General Purpose Input/Output (GPIO) pins**.

For simple digital or analog connections, a sensor might be wired directly to one or more of these pins. However, for more complex digital communication, standardized serial communication protocols, or **buses**, are used. The most common buses mentioned in the lectures are:

- **SPI (Serial Peripheral Interface):** A fast, synchronous communication bus.
- **I2C (Inter-Integrated Circuit):** A simpler, two-wire bus that is also widely used for connecting multiple peripherals to a single microcontroller.

These buses provide a reliable way for the microcontroller to send commands and read data from various sensors and other peripherals.

10. The Accelerometer

An accelerometer is a sensor that measures proper acceleration, which is the acceleration it experiences relative to freefall. It is a foundational sensor in countless CPS applications, from smartphone orientation to vehicle stability control.

Main Issues & Considerations

1. **Saturation:** Every accelerometer has a specific measurement range (e.g., $\pm 2g$, $\pm 4g$, $\pm 16g$). If the device experiences acceleration beyond this range, the sensor's output will be "clipped" or "saturated" at its maximum value. This leads to a loss of information, highlighting the importance of matching the sensor's range to the application's expected forces.
2. **Gravity:** A stationary accelerometer will always detect Earth's gravitational field, measuring $1g$ (9.8 m/s^2) on the axis pointing away from the ground. This constant gravitational vector must be mathematically accounted for and removed if one is interested only in the dynamic acceleration (i.e., motion) of the device.
3. **Noise:** The raw output from an accelerometer is often noisy due to mechanical vibrations and electrical interference. This noise can obscure the true signal and typically must be reduced using digital filtering techniques, such as a low-pass or Kalman filter.
4. **Bias and Drift:** Accelerometers can have a non-zero output even when perfectly still (bias) and this bias can change over time or with temperature (drift). These inaccuracies require periodic calibration to ensure reliable data.

Programming and Usage

Using an accelerometer in a project typically involves the following steps:

1. **Initialization:** The microcontroller communicates with the accelerometer (usually over I2C or SPI) to set key parameters. This includes selecting the measurement range (e.g., $\pm 2g$) and the output data rate (e.g., 100 Hz).
2. **Data Reading:** The main program loop periodically requests data from the accelerometer. The sensor returns three separate raw digital values, representing the acceleration along its X, Y, and Z axes.
3. **Calibration:** To correct for bias, an initial calibration is performed. This involves placing the sensor in several known static orientations (e.g., flat, on each of its six sides) to measure the offset on each axis. These offset values are then subtracted from all future raw readings.
4. **Data Processing & Feature Extraction:** The calibrated data is then used to derive meaningful information. A common step is to calculate the magnitude of the total acceleration vector ($\sqrt{X^2 + Y^2 + Z^2}$). This processed data can then be used as a "feature" for machine learning algorithms to perform tasks like step counting, fall detection, or activity recognition.

11. Smart City IoT Infrastructure Sensors

To build an IoT infrastructure for a smart city, our team would deploy a variety of sensors targeted at improving efficiency, safety, and quality of life across several key domains.

Transportation

Sensors: Magnetometers, inductive loop sensors, and cameras.

Why: These sensors would be embedded in roads and mounted at intersections to detect vehicle presence and traffic flow in real-time. This data would feed into an **adaptive traffic light** system to optimize signal timing, reducing congestion and vehicle emissions. We would also use temperature and humidity sensors to monitor for potentially icy road conditions and alert both authorities and drivers.

Environment

Sensors: Air quality sensors (for NO_2 , O_3), ultrasonic sensors, light sensors, and microphones.

Why: Air quality sensors would create a real-time pollution map of the city, enabling public health alerts and policy decisions. **Ultrasonic sensors** placed inside public trash bins would detect when they are full, enabling a **smart waste management** system that optimizes collection routes, saving fuel and time. Light sensors would control smart streetlights, dimming or turning them off when no one is present to save energy. Microphones would monitor ambient noise levels to enforce noise ordinances.

Public Safety & Infrastructure

Sensors: Cameras with ML capabilities, acoustic sensors (microphones), and accelerometers.

Why: A network of smart cameras can be used to monitor crowds and detect unusual activity. Acoustic sensors can automatically detect the sound of incidents like gunshots or car accidents, pinpointing the location and enabling faster emergency dispatch. Accelerometers mounted on critical infrastructure like bridges and buildings can monitor for structural stress or seismic activity, providing early warnings of potential failures.

12. Sensors for Autonomous Vehicles

Autonomous vehicles require a rich and redundant suite of sensors to perceive their environment, localize themselves accurately, and navigate safely. The principle of **sensor fusion**—combining data from multiple sensor types—is critical to building a system that is robust in all conditions.

The essential sensors for a typical driverless car are:

- **LiDAR (Light Detection and Ranging):** This is a primary sensor for perception. It spins to emit laser pulses, creating a precise, 360-degree, 3D map of the car's surroundings. It is excellent for detecting the shape and distance of objects like other cars, pedestrians, and the road geometry.

- **Radar (Radio Detection and Ranging):** Radar emits radio waves to determine the range, angle, and velocity of objects. Its key advantage is its excellent performance in adverse weather conditions like rain, fog, and snow, where LiDAR and cameras may struggle. It is crucial for features like adaptive cruise control and collision avoidance.
- **Cameras (Optical/Video):** Cameras provide rich, high-resolution visual information that other sensors lack. They are essential for reading traffic signs, detecting the color of traffic lights, identifying lane markings, and classifying objects (e.g., distinguishing a pedestrian from a bicycle).
- **IMU (Inertial Measurement Unit):** An IMU combines an **accelerometer** and a **gyroscope**. It measures the car's orientation, turn rate, and forces. This is vital for understanding the vehicle's dynamics and for tracking its precise position and orientation between GPS updates (a process called dead reckoning).
- **GPS (Global Positioning System):** Provides the vehicle's absolute location on a global map, which is fundamental for navigation from a starting point to a destination. This is typically a high-precision, augmented version of GPS for centimeter-level accuracy.
- **Ultrasonic Sensors:** These sensors use sound waves to detect objects at very close range. They are low-cost and primarily used for low-speed maneuvers like parking and detecting objects in the car's immediate blind spots.

References

- [1] *CS 6762: Signal Processing, Machine Learning and Control Class Materials and Lectures*. University of Virginia.
- [2] Oppenheim, A. V., Schafer, R. W. (2009). *Discrete-Time Signal Processing* (3rd ed.). Pearson. *This is a foundational textbook in digital signal processing that provides a formal and detailed explanation of the Nyquist-Shannon Sampling Theorem and concepts like aliasing.*
- [3] Analog Devices. (2011). *Application Note AN-1057: Using an Accelerometer for Inclination Sensing*. Analog Devices. *This technical note from a major sensor manufacturer details the working principles of MEMS accelerometers, including common issues like bias, noise, and the effect of gravity, which are critical for programming and usage.*
- [4] Badue, C., Guidolini, R., Carneiro, R. V., Azevedo, P., Farias, V. B., De Souza, A. F. (2021). Self-Driving Cars: A Survey. *Expert Systems with Applications*, 165, 113816. *This survey paper provides a comprehensive overview of the sensor suites (LiDAR, Radar, Cameras, IMU, etc.) used in autonomous vehicles and discusses the importance of sensor fusion.*
- [5] Sudevalayam, S., Kulkarni, P. (2011). Energy Harvesting Sensor Nodes: Survey and Implications. *IEEE Communications Surveys Tutorials*, 13(3), 443–461. *A highly cited survey that explains energy harvesting concepts and reviews sources such as solar for enabling long-term operation of IoT and wireless sensor devices.*

- [6] SparkFun Electronics. (n.d.). *Serial Peripheral Interface (SPI) and I²C*. SparkFun. *These tutorials provide practical explanations of the SPI and I2C communication protocols, detailing how microcontrollers use these buses to connect with and control peripherals like sensors.*

Acknowledgment

This Homework makes use of Google Search, Perplexity and Gemini for research to form and refine answers as required for this homework and some of the main sources for the information are cited. The author acknowledges that they have not given or received unauthorized assistance on this homework. The Author also acknowledges that they have understood the course content and lectures for the preparation of these homework solutions.